Guia Pratico de Nomenclatura para Codigo

Nomenclatura clara para projetos TypeScript / JavaScript

Um guia direto para nomear variaveis, metodos, classes, contratos e arquivos.

Foco: legibilidade, intencao e consistencia em Clean Architecture, Hexagonal, DDD e CQRS.

Regra central: o nome deve revelar o papel do codigo sem exigir abrir a implementacao.

Exemplo ruim: const data = await repo.find(id)

```
const product = await productRepository.findById(productId);
```

Use este material como referencia rapida durante implementacao, review e refatoracao.

Sumario

1. Fundamentos de nomes

2. Entidades, VO e Use Cases

3. Controllers, DTOs, Repositories e Gateways

4. Strategies, Observers, Factories e Presenters

5. Infraestrutura e argumentos de construtor

6. CQRS, eventos, erros e cheat sheet

1. Fundamentos de nomes

Variaveis

Prefira substantivos que indiquem exatamente o conteudo.

```
product | activeProducts | authenticatedUser | totalProducts
```

Evite: data, value, result, obj, temp, item quando o contexto permite ser especifico.

Booleanos

Devem soar como perguntas.

```
isActive | hasPermission | canDelete | shouldRetry | exists
```

Metodos e funcoes

Normalmente comecam por verbo e descrevem uma acao.

```
createProduct() | findById() | calculatePrice() | sendEmail()
```

findById: pode nao encontrar. getById: geralmente pressupoe existencia ou lanca erro.

Colecoes, IDs e datas

```
product / products | productId | createdAt | dueDate | totalItems
```

Use plural para colecoes; sufixo Id para identificadores; At para instante; Date para data conceitual.

2-3. Dominio, aplicacao e portas

Entidades

Use substantivos do dominio. A pasta ja pode informar que e uma entidade.

```
Product | Customer | Order | Payment
```

Value Objects

Nomeie o conceito, nao a categoria arquitetural.

```
Email | Money | CPF | ProductId | Address
```

Use Cases

Padrao forte: Verbo + Objeto.

```
CreateProduct | UpdateProduct | FindProductById | CancelOrder
```

Controllers

```
ProductController | CreateProductController
```

DTOs

Inclua a finalidade ou direcao.

```
CreateProductDTO | UpdateProductDTO | ProductResponseDTO
```

Repositories

Contrato no dominio/aplicacao; tecnologia na implementacao.

```
ProductRepository -> PrismaProductRepository
```

Gateways

Representam comunicacao com recursos ou servicos externos.

```
PaymentGateway -> StripePaymentGateway
```

```
EmailGateway -> ResendEmailGateway
```

4-5. Comportamentos, apresentacao e infraestrutura

Strategies

Nomeie o comportamento variavel.

`CompressionStrategy -> ZipCompressionStrategy`

`ExtractionStrategy -> PdfExtractionStrategy`

Observers

Pode usar o evento observado ou a acao disparada pelo evento.

`OrderCreatedObserver | SendEmailOnUserRegistered`

Factories, Mappers, Presenters e Validators

`ProductFactory | ProductMapper | ProductPresenter | ProductValidator`

Services

Evite ProductService quando existir uma responsabilidade mais especifica.

`PasswordHashService | TokenService | ImageProcessingService`

Infraestrutura

Explicite tecnologia + papel quando isso melhora a leitura.

`prisma-client.ts | redis-cache.ts | resend-email-gateway.ts`

Argumentos de construtor

Nomeie pela responsabilidade exercida na classe.

`productRepository | paymentGateway | tokenProvider | cache`

Evite abreviacoes desnecessarias como repo, svc e mgr em codigo de negocio.

6. CQRS, eventos, erros e cheat sheet

Commands e Queries

```
CreateProductCommand | UpdateProductCommand | DeleteProductCommand
FindProductByIdQuery | ListProductsQuery
CreateProductCommandHandler | ListProductsQueryHandler
```

Eventos

Eventos representam fatos que ja aconteceram; use passado.

```
UserCreated | OrderPlaced | OrderPaid | PaymentConfirmed
```

Errors

Nomeie a condicao concreta de erro.

```
ProductNotFoundError | InvalidProductPriceError | EmailAlreadyExistsError
```

Cheat sheet

Coisa: Product, Order, Email, Money

Acao: CreateProduct, DeleteOrder, SendEmail

Contrato: ProductRepository, PaymentGateway, Cache

Implementacao: PrismaProductRepository, StripePaymentGateway, RedisCache

Transformacao: ProductMapper, ProductPresenter

Regra: ProductValidator, DiscountStrategy

Evento: ProductCreated, OrderPaid

Entrada/saida: CreateProductDTO, ProductResponseDTO

Se a classe so aceita nomes como Manager, Helper, Utils ou Service, revise se ela tem responsabilidades c